

RISC-V 的 Store AMO access fault 偵錯實例

原創 qinyunti [嵌入式 Lee](#) 2023 年 05 月 10 日 10:10 湖南

前言

本文以一個實例分享 RISC-V 的 Store AMO access fault 異常的偵錯過程。
Store AMO access fault 主要發生在非法地址訪問時(棧溢出,指針異常等)。

過程

現像是程式執行時進入了異常中斷,如下

使用 bt 回溯可以看到進入了異常處理函數 exception ()

```
(gdb) bt
.....
#6 0x02002eb2 in exception () at src/lib/riscv/src/exception.c:55
#7 0x00000000 in ?? ()
Backtrace stopped: frame did not save the PC
(gdb)
```

那麼首先想到的是確認異常原因

查看異常暫存器:**info reg mcause**

可以看到異常原因是 0x07

```
gdb>>> info reg mcause
mcause          0x7          7
```

對應的是 Store/AMO access fault 異常,這是一個寫資料時的異常。

Interrupt	Exception Code	Description
1	0	<i>Reserved</i>
1	1	Supervisor software interrupt
1	2	<i>Reserved</i>
1	3	Machine software interrupt
1	4	<i>Reserved</i>
1	5	Supervisor timer interrupt
1	6	<i>Reserved</i>
1	7	Machine timer interrupt
1	8	<i>Reserved</i>
1	9	Supervisor external interrupt
1	10	<i>Reserved</i>
1	11	Machine external interrupt
1	12–15	<i>Reserved</i>
1	≥ 16	<i>Designated for platform use</i>
0	0	Instruction address misaligned
0	1	Instruction access fault
0	2	Illegal instruction
0	3	Breakpoint
0	4	Load address misaligned
0	5	Load access fault
0	6	Store/AMO address misaligned
0	7	Store/AMO access fault
0	8	Environment call from U-mode
0	9	Environment call from S-mode
0	10	<i>Reserved</i>
0	11	Environment call from M-mode
0	12	Instruction page fault
0	13	Load page fault
0	14	<i>Reserved</i>
0	15	Store/AMO page fault
0	16–23	<i>Reserved</i>
0	24–31	<i>Designated for custom use</i>
0	32–47	<i>Reserved</i>
0	48–63	<i>Designated for custom use</i>
0	≥ 64	<i>Reserved</i>

Table 3.6: Machine cause register (mcause) values after trap.

那麼接下來就要確認寫哪個地方的資料錯誤了呢？我們可以通過 `info reg mtval` 查看 `mtval` 暫存器看到。寫 `0x28382ad0` 時產生了異常。

```
gdb>>> info reg mtval
mtval          0x28382ad0      64
```

那麼什麼時候寫這個地址導致了異常呢，之前 `bt` 已經看不到回溯的地方了。

我們可以使用資料斷點來監控

設定資料斷點 `watch *(unsigned int*)0x28382ad0`

重新載入程式執行 `load`

`c` 運行

看到斷點停在了 `memset` 函數處，這印證了我們前面分析的是寫資料導致的問題

```
(gdb) watch *(unsigned int*)0x28382ad0
Hardware watchpoint 6: *(unsigned int*)0x28382ad0
(gdb) load
Loading section .init, size 0x80 lma 0x20000000
Loading section .iram_text, size 0x60fc lma 0x20000080
Loading section .text, size 0x1f8d6 lma 0x2000617c
Loading section .icache_text, size 0x109a lma 0x20025a52
Loading section .rodata, size 0x4eb4 lma 0x20026af0
Loading section .data, size 0x5e6b4 lma 0x2002b9a8
Start address 0x20000000, load size 565332
Transfer rate: 55 KB/sec, 14495 bytes/write.
(gdb) c
Continuing.

Program received signal SIGTRAP, Trace/breakpoint trap.
0x02000530 in memset ()
(gdb) █
```

繼續 `bt` 查看

```
(gdb) bt
#0 0x02000530 in memset ()
#1 0x20025b2a in prvInitialiseNewTask (xRegions=0x0, pxNewTCB=0x28383
#2 xTaskCreate (pxTaskCode=pxTaskCode@entry=0x2000a7fc <wrapper_task_
#3 0x20026668 in os_create_task_ext (fn=0x2000b248 <app_entry>, arg=a
#4 0x20006392 in main () at src/project/custom_app/core0/main.c:258
(gdb) █
```

我們找到對應的程式碼處

```

static void ICACHE_ATTR prvInitialiseNewTask( TaskFunction_t pxTaskCode,
const char *const pcName, /*lint !e971 Unqualified char types are allowed for strings and single characters only*/
const uint32_t ulStackDepth,
void *const pvParameters,
UBaseType_t uxPriority,
TaskHandle_t *const pxCreatedTask,
TCB_t *pxNewTCB,
const MemoryRegion_t *const xRegions )
{
    StackType_t *pxTopOfStack;
    UBaseType_t x;

    #if( portUSING_MPU_WRAPPERS == 1 )
        /* Should the task be created in privileged mode? */
        BaseType_t xRunPrivileged;
        if ( ( uxPriority & portPRIVILEGE_BIT ) != 0U ) {
            xRunPrivileged = pdTRUE;
        } else {
            xRunPrivileged = pdFALSE;
        }
        uxPriority &= ~portPRIVILEGE_BIT;
    #endif /* portUSING_MPU_WRAPPERS == 1 */

    /* Avoid dependency on memset() if it is not required. */
    #if( tskSET_NEW_STACKS_TO_KNOWN_VALUE == 1 )
    {
        /* Fill the stack with a known value to assist debugging. */
        ( void ) memset( pxNewTCB->pxStack, ( int ) tskSTACK_FILL_BYTE, ( size_t ) ulStackDepth * sizeof( StackType_t ) );
    }
    #endif /* tskSET_NEW_STACKS_TO_KNOWN_VALUE */

    /* Calculate the top of stack address. This depends on whether the stack
    grows from high memory to low (as per the 80x86) or vice versa.
    portSTACK_GROWTH is used to make the result positive or negative as required

```

看下 `memset` 的參數，如果看不到可以在 `memset` 前打斷點重新運行

```

(gdb) hb tasks.c:860
Hardware assisted breakpoint 1 at 0x20025b1c: file src/os/freertos_10_2_1/freertos/tasks.c
(gdb)

```

最終確認確實是棧初始化時寫 `0x28382ad0` 這個地址的內容錯誤，原因是不具備寫屬性導致異常

```

(gdb) p pxStack
$8 = (StackType_t *) 0x28382ad0
(gdb)

```

```

(gdb) set *(unsigned int*)0x28382ad0=0x00
Cannot access memory at address 0x28382ad0
(gdb)

```

當然為什麼這個地址不能寫和這裡沒關係了，是另外一回事了，是我們的環境 `DDR` 的問題。

總結

以上是一個分析實例的過程，遇到類似問題可以參考。有幾個關鍵點一是確認異常原因，異常訪問的地址，然後通過資料斷點確認什麼時候訪問了這個地址，到此基本就確認問題了，後面就順藤摸瓜了。